

FACETS: A MULTI-DIMENSIONAL FRAMEWORK FOR ASSESSING LLMs ON SOFTWARE ENGINEERING TASKS

By

Yash Patel

A SENIOR RESEARCH PAPER PRESENTED TO THE DEPARTMENT OF
MATHEMATICS AND COMPUTER SCIENCE OF STETSON UNIVERSITY IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE

STETSON UNIVERSITY

2026

Acknowledgments

We thank the Stetson University Department of Mathematics and Computer Science for providing API access and compute resources used in this work. We are grateful to Dr. Basar Koc for his guidance and mentorship throughout this research. We also acknowledge the use of Claude Code (Anthropic) as a coding assistant during development.

Contents

1	Introduction	7
2	Related Work	8
2.1	Function Level Benchmarks	8
2.2	Software Engineering Benchmarks	9
2.3	Static Analysis and LLM-as-Judge Frameworks	9
3	Problem Definition	10
4	System Design and Architecture	10
4.1	Core Components	11
4.2	Providers	13
4.3	Benchmark Tasks and Datasets	14
4.4	Evaluation Dimensions	15
4.5	Design Choices	18
4.5.1	Modularity and Separation of Concerns	18
4.5.2	Benchmark Matrix and Scoring Hierarchy	19

4.6	Execution and Usage	20
5	Experimentation and Results	21
5.1	Experimental Setup	22
5.2	Models Evaluations and Scores	22
5.3	Result Analysis	23
5.4	Task Performance Breakdown	26
5.5	Comparison	27
6	Discussion and Limitations	28
7	Conclusion and Future Work	30

List of Tables

1	Benchmark to Dimension Mapping and Weights	20
2	Ollama Hardware Specifications	22
3	Models Evaluated	23
4	FACETS vs. LiveCodeBench	28

List of Figures

1	Benchmark Performance Radar	23
2	Violin plot depicting individual prompt performance scores	24
3	Scatter plot of model performance scores vs. model size (in billions of parameters)	25
4	Mean task performance scores across all models	27

Abstract

FACETS: A Multi-Dimensional Framework for Assessing LLMs on Software Engineering Tasks

By

Yash Patel

December 2026

Advisor: Dr. Basar Koc

Department: Mathematics and Computer Science

LLMs have become central to modern software development, yet existing benchmarks evaluate them narrowly on functional correctness, overlooking dimensions such as style, security, structural fidelity, and runtime efficiency. This paper presents FACETS (Framework for Assessing Code across Evaluation Tasks and Scores), a modular multi-dimensional benchmarking framework that evaluates code generation models across six software engineering tasks, code generation, bug fixing, code review, refactoring, test generation, and cross language translation, using nine independent quality dimensions aggregated through a weighted scoring hierarchy. We evaluated 13 LLMs spanning parameter counts from 3B to 70B and frontier scale. Closed source frontier models led the rankings, with Claude Opus 4.6 achieving the highest final score of 89.33%, while the weakest model scored 64.56%. Across all models, performance was strongest on structurally defined tasks and weakest on refactoring and test generation, revealing a persistent gap between syntactic fluency and semantic understanding.

1 Introduction

Large Language Models (LLMs) have been a cornerstone of Generative AI, since their inception in the early 2010s. These models have evolved from simple architectures to complex systems capable of understanding and generating human-like text. In the past few years, every industry has found a unique and tailored use for LLMs

Arguably, the most remarkable use of LLMs has been in the field of software development, where they have been used in every aspect of the software development lifecycle, from system design to code generation, testing and maintainance. Among all the applications of LLMs in software development, one of the most prominent and impactful is the use of LLMs in code generation [3]. Code generation models represent a transformative application of LLMs in software development, automating tasks ranging from snippet generation to full function implementation. The landscape of code generation is bifurcated into two distinct categories: open source models and closed source frontier models. Open source code generation models, such as Qwen3-Coder-Next [4], GLM-5 [5], and Kimi-K-2.5 [6], provide accessibility and customization opportunities for researchers and developers, enabling fine tuning on domain specific datasets and transparency in model architectures. Conversely, closed source frontier models including GPT-5.3-Codex [7], Claude-Opus-4.6 [8], and Gemini-3.1-Pro [9] have demonstrated state-of-the-art performance on many benchmarks, though they come with limitations in terms of accessibility and transparency.

As the number of code generation models continues to grow and their capabilities become increasingly sophisticated, a fundamental question arises: how do these models compare against one another, and what metrics should be used to evaluate their performance? The evaluation of code generation models is inherently more complex than that of general purpose LLMs, as

generated code must not only be syntactically correct but also functionally accurate, efficient, secure, and maintainable. Traditional natural language evaluation metrics such as BLEU and ROUGE have proven to be insufficient for capturing the nuanced quality of generated code [10], prompting the development of specialized benchmarks such as HumanEval [11], Most Basic Python Problems (MBPP) [12], and SWE-bench [13], which evaluate models on tasks ranging from standalone function synthesis to real world software engineering problems.

2 Related Work

2.1 Function Level Benchmarks

Early benchmarks for code generation models focused on function level synthesis, where models were tasked with generating a single function based on a natural language description. HumanEval [11] and MBPP [12] are two of the most widely used benchmarks in this category, evaluating models on their knowledge of algorithms, standard library usage, and problem solving abilities. The pass@ k metric [11], introduced alongside HumanEval, became the de facto standard in this paradigm, measuring the probability that at least one of k generated samples passes all unit tests. While being widely adopted, it introduces its own set of challenges, most notably the requirement for comprehensive and unbiased test suites, which are expensive to construct and maintain.

2.2 Software Engineering Benchmarks

To address the limitations of function level benchmarks, more recent efforts have focused on software engineering benchmarks that evaluate models on their ability to solve real world software engineering problems. SWE-bench [13] is a notable example of this category, where models are evaluated on their ability to resolve actual GitHub issues, which often require understanding of existing codebases, debugging skills, and the ability to generate code that integrates seamlessly with existing systems. These benchmarks provide a more holistic evaluation of code generation models, but they also introduce new challenges in terms of evaluation criteria and reproducibility while still primarily relying on functional correctness as the main evaluation criterion.

2.3 Static Analysis and LLM-as-Judge Frameworks

More recently, static analysis based approaches have emerged as a complementary evaluation dimension, assessing generated code on properties such as cyclomatic complexity, code smells, and adherence to style guidelines, qualities that execution based metrics fail to capture [14]. Furthermore, with the rapid advancement of LLMs themselves, LLM-as-judge frameworks [15] have gained traction as a scalable alternative, where a powerful model evaluates the quality, readability, and correctness of generated code. While promising, these approaches raise concerns around evaluator bias and reproducibility. Taken together, the evolution of evaluation methodologies reflects a growing recognition that functional correctness alone is an insufficient proxy for code quality in real world software engineering contexts.

3 Problem Definition

Despite the availability of these benchmarks, significant gaps remain in the evaluation landscape. Most existing benchmarks focus narrowly on functional correctness, often measured through unit test pass rates, while neglecting equally important dimensions such as code efficiency, readability, adherence to best practices, and performance across diverse problem domains. Furthermore, the rapid pace of model development has led to a fragmented evaluation ecosystem, where different models are assessed under varying conditions, making direct and fair comparisons difficult.

This paper aims to address these gaps by presenting a comprehensive and systematic benchmarking framework. We evaluate a representative set of models across multiple dimensions including functional correctness, computational efficiency, semantic correctness, and code quality, using a combination of established benchmarking datasets and novel evaluation criteria. Our goal is to provide an empirical foundation that enables researchers and practitioners to make informed decisions when selecting code generation models for their specific needs, while also identifying the strengths, limitations, and areas for improvement across the current generation of models.

4 System Design and Architecture

FACETS is designed as a modular, extensible evaluation framework that decouples the four core concerns of LLM evaluation: model access, task specification, quality measurement, and result aggregation. The system accepts a model configuration as input, executes a set of predefined benchmark tasks against multiple datasets of code samples, evaluates each

inference response across multiple quality dimensions, and produces a structured score report.

At the highest level, the framework follows a pipeline architecture with four stages:

- **Model Access Layer:** This layer abstracts the interaction with different code generation models, providing a unified interface for sending prompts and receiving generated code. It supports both open source models, which can be run locally, and closed source frontier models, which are accessed through APIs.
- **Task Execution Layer:** In this layer each benchmark record is submitted to the model and a raw response is collected. The framework supports a variety of task types, and can be easily extended to accommodate new benchmarks and task formats.
- **Dimension Evaluation:** Here, the generated code is assessed across up to five quality dimensions depending on the task.
- **Result Aggregation Layer:** This layer combines the dimension scores into task level and suite level scores using a weighted hierarchy.

The framework is designed using `python` and is invoked either through a headless command-line interface that supports both interactive and configuration driven modes, or through a terminal user interface for interactive session.

4.1 Core Components

We have organized the framework into five primary modules, each responsible for a specific aspect of the evaluation process:

Core (`core/`) – contains abstract base classes (`BaseProvider`, `BaseBenchmark`, `BaseDimension`), the shared data structures (`LLMResponse`, `BenchmarkResult`, `DimensionResult`, `ModelConfig`), the `TestSuite` orchestrator, the `ScoringEngine` aggregator, and the `ProviderRegistry` factory.

Providers (`providers/`) – contains one subdirectory per LLM provider, each implementing the `BaseProvider` interface. All providers auto register with the registry via a decorator, making them available through a unified `ProviderRegistry.create(ModelConfig)` call. We have implemented providers for any model available through `huggingface`, `OpenAI`, `Anthropic` and `Ollama`.

Benchmarks (`benchmarks/`) – contains two sub modules: `tasks/`, which defines the six benchmark task types, and `dimensions/`, which defines the nine evaluation dimensions. A central `matrix.py` module specifies which dimensions apply to which tasks and the associated scoring weights.

Datasets (`datasets/`) – contains the `DatasetMapper` class and the CSV dataset files organized by task and programming language. The mapper handles column normalization, record loading, and the translation of raw CSV rows into task specific keyword arguments.

Utils (`utils/`) – provides a language agnostic `CodeRunner` that can execute Python, C++, and JavaScript code in subprocess-isolated environments, used by functional correctness and runtime analysis dimensions.

4.2 Providers

The framework supports four LLM provider backends, each implementing a common interface with three methods: `connect()`, which initializes the client connection and validates credentials; `complete(prompt, system_prompt)`, which submits a prompt and returns a standardized `LLMResponse`; and `is_available()`, which checks service reachability before execution begins.

OpenAI (`OpenAIProvider`) uses the OpenAI Chat Completions API and supports both chat and legacy completion endpoints. It accepts standard parameters including `model_name`, `temperature`, `max_tokens`, and `base_url`, the latter enabling compatibility with OpenAI-compatible third-party endpoints.

Anthropic (`AnthropicProvider`) uses the Anthropic Messages API. It accepts a `model_name`, `temperature`, and `max_tokens`. The provider returns input and output token counts as part of the standardized response.

Ollama (`OllamaProvider`) communicates with a locally running Ollama server via its REST API. It requires no external API key, making it the preferred backend for evaluating open-source models in offline or resource-constrained environments. It exposes a `list_local_models()` method for introspecting which models are available on the host machine.

HuggingFace (`HuggingFaceProvider`) runs models directly using the `transformers` library via an `AutoModelForCausalLM` pipeline. It accepts a model identifier (Hub ID or local path), an optional HuggingFace token for gated models, and advanced loading parameters including `device_map`, `torch_dtype`, `load_in_8bit`, and `load_in_4bit`, enabling quantized inference on consumer hardware.

All four providers produce a unified `LLMResponse` object containing the generated text, model name, provider name, token counts, latency in milliseconds, and an optional error field, which allows all downstream evaluation logic to operate provider-agnostically.

4.3 Benchmark Tasks and Datasets

FACETS defines six benchmark task types, each corresponding to a distinct software engineering activity. All tasks inherit from `BaseBenchmark`, which mandates two abstract methods: `build_prompt()`, which constructs the task-specific prompt from pre-processed csv dataset records, and `run()`, which executes the LLM call and orchestrates dimensional evaluation.

Code Generation evaluates a model’s ability to synthesize a complete Python function from a natural language description. Each dataset record supplies a task description, a unit test suite, and the name of the function under test. The dataset comprises of 164 code samples drawn from OpenAi’s HumanEval [11].

Bug Fixing assesses a model’s ability to identify and correct a fault in a provided Python snippet. Each record, adapted from QuixBugs dataset [17], supplies the original buggy code to be fixed to the LLM. The generated code is then compared against a known correct expected output.

Code Review measures the quality of a model’s natural-language review of a given code snippet against a reference review from the dataset, composed of 20 code samples coupled with their reviews synthetically generated by prompting powerful frontier models.

Refactoring assesses a model’s ability to restructure a Python function according to a specified refactoring goal while preserving its observable behavior. 20 synthetically generated

dataset samples include the original code, a free-text description of the desired transformation, and a fixed test harness that exercises the function.

Test Generation evaluates a model’s ability to produce a meaningful unit test suite for a given Python function. The test generation dataset consists of 20 records, carefully curated from the HumanEval dataset [11], each containing a function implementation, its natural language description, and an entry point for testing.

Translation assesses a model’s ability to faithfully translate a `javascript` code snippet into a `python` implementation while preserving its runtime behavior. Here, the dataset consists of 20 pairs of semantically equivalent code snippets in `javascript` and their outputs, synthetically generated by prompting powerful frontier models.

4.4 Evaluation Dimensions

We define nine evaluation dimensions, each implemented as an independent class inheriting from `BaseDimension`. Every dimension exposes a single `evaluate(language, generated_code, original_code, **kwargs)` method that returns a `DimensionResult` with a normalized score in $[0.0, 1.0]$, a pass/fail flag, and structured diagnostic details.

Functional Correctness (FC) verifies that generated code preserves the runtime behavior of the original. The evaluation path depends on the task. For *refactoring*, the generated code is appended with a fixed test harness and executed; its standard output is compared against a known-good expected output. For *bug fixing* and *translation*, the generated code is executed directly and its output is compared against a stored expected string. For all other tasks, both the original and generated code are executed independently in language-appropriate

subprocess runners and their standard outputs are compared. Before comparison, outputs are normalized to strip insignificant whitespace and reconcile language-specific literal differences. A score of 1.0 is awarded when normalized outputs match exactly and 0.0 otherwise.

Code Consistency (CC) evaluates whether the generated code maintains uniform stylistic conventions throughout. Three properties are checked statically via regular expression analysis: indentation consistency (spaces versus tabs must not be mixed), naming convention consistency (camelCase and snake_case identifiers must not co-exist), and string quote consistency (single and double quotes must not appear in proportions that diverge by more than 30%). Each failed check contributes equally to an issue count, and the final score is $1 - \frac{\text{issues}}{3}$, clipped to $[0, 1]$.

Linting measures whether the generated code introduces new style or error violations relative to the original. Both the original and generated code are written to temporary files and analyzed with `pylint` (warnings and errors only, with all other checks disabled). If the generated code has no more violations than the original, the score is 1.0. If it introduces new violations, the score is penalized proportionally: $\max\left(0, 1 - \frac{\Delta_{\text{violations}}}{\max(\text{orig_violations}, 1)}\right)$.

Vulnerabilities checks whether the generated code introduces new security issues relative to the original. Both versions are written to temporary files and scanned with `bandit`, which reports issues at various severity levels as a raw count. If the generated code has fewer or equal vulnerabilities, the score is 1.0. If it introduces new ones, the score is penalized as $\max\left(0, 1 - \frac{\Delta_{\text{vulns}}}{\max(\text{gen_vulns}, 1)}\right)$.

Semantic Drift (SD) quantifies how much the structure of the generated code diverges from the original using Tree Edit Distance (TED). Both code snippets are parsed into their Abstract Syntax Trees, serialized into a bracket notation, and compared using the APTED

algorithm [16]. The raw edit distance is normalized by the combined size of both trees to produce a similarity score in $[0, 1]$, where 1.0 indicates structurally identical trees and lower values reflect greater divergence. Parse failures are handled gracefully using a sequence of fallback strategies (dedenting, function wrapping, class wrapping) before resorting to a score of 0.0.

Runtime Analysis (RA) measures whether the generated code is faster and more memory-efficient than the original. Both snippets are executed inside an instrumented subprocess that records wall-clock time and peak memory usage. For each metric, the ratio of generated to original is computed and mapped through a logistic function $\sigma(x) = \frac{1}{1+e^{3\ln(x)}}$, which yields 0.5 for equal performance, approaches 1.0 for large improvements, and approaches 0.0 for large regressions. The final score is the mean of the time and memory scores. For refactoring tasks, the gold expected refactored code serves as the performance baseline instead of the original.

Code Review Quality (CRQ) measures the semantic similarity between the model-generated review and a reference review from the dataset. Both texts are embedded using the `nomic-embed-text` model served locally via Ollama, and their cosine similarity is computed as the score. This embedding-based approach captures paraphrastic agreement and shared technical concepts rather than requiring lexical overlap with the reference.

Code Generation Tests (CGT) evaluates generated test suites against the original correct implementation using two complementary signals. First, the generated tests are executed against the original code using `pytest`; the fraction of tests that pass is the *pass rate*. Second, a null mutant is constructed by appending a redefinition of the function under test that unconditionally returns `None`, and the passing tests are re-run against it; the fraction of previously-passing tests that now fail is the *kill rate*. The final score is $\text{pass_rate} \times \text{kill_rate}$,

which rewards test suites that are both correct and sensitive to implementation errors. When no entry point is provided, mutation testing is skipped and the score falls back to pass rate alone.

Test Pass Rate (TPR) evaluates the quality of model-generated test suites against the original correct implementation using two complementary signals. First, the generated tests are executed against the original code using `pytest`; the fraction of tests that pass is the *pass rate*, filtering out tests that are simply wrong. Second, a null mutant is constructed by appending a redefinition of the function under test that unconditionally returns `None`, and the previously-passing tests are re-run against it; the fraction that now fail is the *kill rate*. The final score is $\text{pass_rate} \times \text{kill_rate}$, penalizing both incorrect and trivially weak tests. When no entry point is provided, mutation testing is skipped and the score falls back to pass rate alone.

4.5 Design Choices

4.5.1 Modularity and Separation of Concerns

FACETS is architected around strict separation of concerns at every layer. Providers, tasks, dimensions, datasets, and the scoring engine are each independently developed, tested, and extensible. This modularity offers three principal advantages in the context of a benchmarking system.

First, it enables provider agnostic evaluation: because all providers implement a common interface, any task can be run against any model without modification to the task logic. Second, it enables dimension agnostic tasks: because dimensions are instantiated at runtime

from the benchmark matrix rather than hard-coded into task logic, the set of quality criteria applied to a task can be extended without touching task implementations. Third, it enables reproducibility: because dataset loading, prompt construction, LLM invocation, and evaluation are each isolated in separate components, any single stage can be swapped, logged, or replayed independently.

This separation also supports practical research workflows. A researcher adding a new evaluation dimension need only implement `BaseDimension` and add an entry to the benchmark matrix; the remainder of the framework requires no changes. Similarly, adding support for a new LLM provider requires only implementing `BaseProvider` and applying the registration decorator.

4.5.2 Benchmark Matrix and Scoring Hierarchy

The benchmark matrix serves as the central configuration that binds each task to its evaluation dimensions and their relative weights. It maps each of the six task names to an ordered list of dimension classes and, assigns each dimension a scalar weight such that weights within a task sum to 1.0. At evaluation time, each task’s `run()` method queries the matrix to discover which dimensions apply and what weight each contributes to the combined score, keeping task logic decoupled from the choice of evaluation criteria. Table 1 illustrates which dimensions are applied to each task and their associated weights in the score aggregation.

Furthermore, the scoring engine implements another hierarchical aggregation strategy to individual benchmark scores. A final score is computed as a weighted sum after applying a final benchmark weighting scheme, which assigns different importance to each task based on its perceived relevance to real world software engineering. This hierarchy allows for nuanced

Table 1: Benchmark to Dimension Mapping and Weights

Benchmark	FC	CC	Lint.	Vuln.	SD	RA	CRQ	CGT	TPR
Bug Fixing	0.70	0.10	0.10	0.05	0.05	-	-	-	-
Code Generation	-	0.40	-	-	-	-	-	0.60	-
Code Review		-	-	-	-	-	1.00	-	-
Refactoring	0.40	0.10	0.10	0.05	-	0.35	-	-	-
Test Generation	-	0.25	-	-	-	-	-	-	0.75
Translation	0.50	0.30	0.10	0.10	-	-	-	-	-

evaluation that reflects the multifaceted nature of code quality, while also providing a single scalar score for easy comparison across models. The final score weights are as follows: Code Generation (**0.27**), Translation (**0.16**), Bug Fixing (**0.15**), Code Review (**0.15**), Refactoring (**0.15**), Test Generation (**0.12**). Together, they sum up to 1.00, reflecting the relative importance of each task in the overall evaluation framework.

4.6 Execution and Usage

The framework provides two user interfaces for executing benchmarks: a headless command-line runner for scripted and batch evaluation, and a terminal user interface (TUI) for interactive exploration and real-time monitoring.

The primary entry point is `run_experiment.py`, a headless CLI script that accepts a configuration file, interactively collects model and suite parameters, selects benchmarks, and executes the evaluation pipeline. The script supports a `YAML` configuration file (default: `config/model_config.yaml`) that specifies the LLM provider, model name, API credentials, and generation parameters (temperature, max tokens). When invoked with the `-yes` flag, the script runs in non-interactive mode, using configuration file defaults without prompting; when invoked without this flag, the script interactively confirms or solicits each setting,

enabling quick parameter changes without file editing. Additional command-line flags allow specifying which benchmarks to run (`-tasks`), the number of dataset records to evaluate per task (`-limit`), the number of independent samples per record for $\text{pass}@k$ estimation (`-num-samples`), and the specific k values to report (`-pass-k`). Once configured, the script instantiates the target provider via `ProviderRegistry.create()`, initializes the `TestSuite` orchestrator with the selected benchmarks and language configuration, calls `suite.run_all()` to execute the full evaluation pipeline, and writes a JSON report with per-record results and aggregated scores to the output directory specified in the configuration. Progress is printed to stdout in real time, including per-benchmark completion status and a final summary with task scores, category-level scores, a weighted final score, and the corresponding letter grade.

For interactive sessions and visual monitoring, the TUI (`tui/app.py`) provides a text-based dashboard displaying real-time progress, dimension-by-dimension breakdowns, and cumulative score updates as benchmarks complete. This interface is particularly useful for exploratory evaluation and qualitative inspection of model outputs.

5 Experimentation and Results

To test the effectiveness and classification accuracy of our benchmarking framework, we ran a series of experiments on a representative set of code generation models, including both open source and closed source frontier models. We discuss the experimental setup, the results obtained across different benchmarks and dimensions, and the insights derived from the analysis.

5.1 Experimental Setup

We tested closed and open source models from the three major providers: OpenAI, Anthropic, and Ollama. For OpenAI and Anthropic, we accessed their models through API calls using the `OpenAIProvider` and `AnthropicProvider` implementations, respectively. For Ollama, we ran local instances of open source models using the `OllamaProvider` on our local machine. The hardware specifications for the machine are listed in Table 2.

Table 2: Ollama Hardware Specifications	
Component	Specification
CPU	Intel i9-13900K
RAM	128GB DDR4
GPU 0	NVIDIA GeForce RTX 4090 (24 GB VRAM)
GPU 1	NVIDIA GeForce RTX 4090 (24 GB VRAM)
Operating System	Ubuntu 24.04.3 LTS

5.2 Models Evaluations and Scores

We have evaluated the following models listed in Table 3 on FACETS. the output temprature was set to 0.7 for all models to ensure deterministic outputs, and the maximum token limit was set to 4096 tokens for all models to accommodate the complexity of the benchmark tasks.

The results indicate that closed source frontier models generally outperform open source models across all benchmarks and dimensions, with the top three performers being closed source models from Anthropic and OpenAI. However, some open source models, particularly those with larger parameter counts and newer architectures, also achieved competitive scores.

Table 3: Models Evaluated			
Model Name	Provider	Total Parameters	Final Score
claude-opus-4.6	Anthropic	undisclosed	89.33%
claude-sonnet-4.6	Anthropic	undisclosed	88.40%
claude-haiku-4.6	Anthropic	undisclosed	86.85%
gpt-5-codex	OpenAI	undisclosed	88.62%
gpt-5.4	OpenAI	undisclosed	88.11%
gpt-5-nano	OpenAI	undisclosed	86.36%
deepseek-r1	Ollama	32B	85.13%
qwen3.6	Ollama	35B	85.20%
gpt-oss	Ollama	20B	83.24%
ministral-3	Ollama	3B	79.48%
llama3	Ollama	70B	79.29%
mistral	Ollama	7B	70.70%
phi4-mini	Ollama	3.8B	64.56%

5.3 Result Analysis

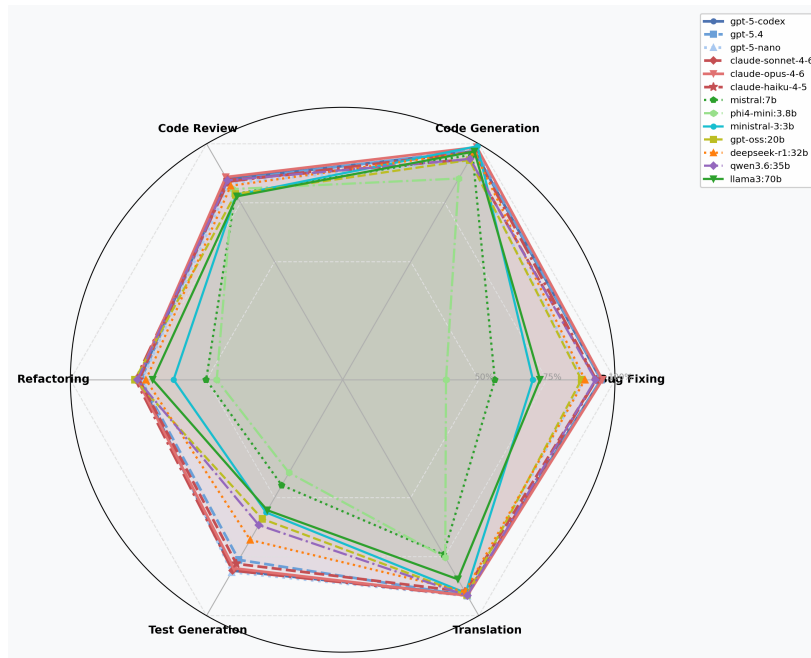


Figure 1: Benchmark Performance Radar

Figure 1 shows the performance of all tested models across the six benchmarks. As a general

trend across all the models, the hexagonal radar plot appears to be skewed towards the right, indicating that the models lack proficiency in refactoring tasks compared to bug fixing and code generation. Visually it can be confirmed that the top three models (claude-opus-4.6, gpt-5-codex, and claude-sonnet-4.6) have a more balanced performance across all benchmarks and the worst performing benchmarks for all models are refactoring, code review, and test generation.

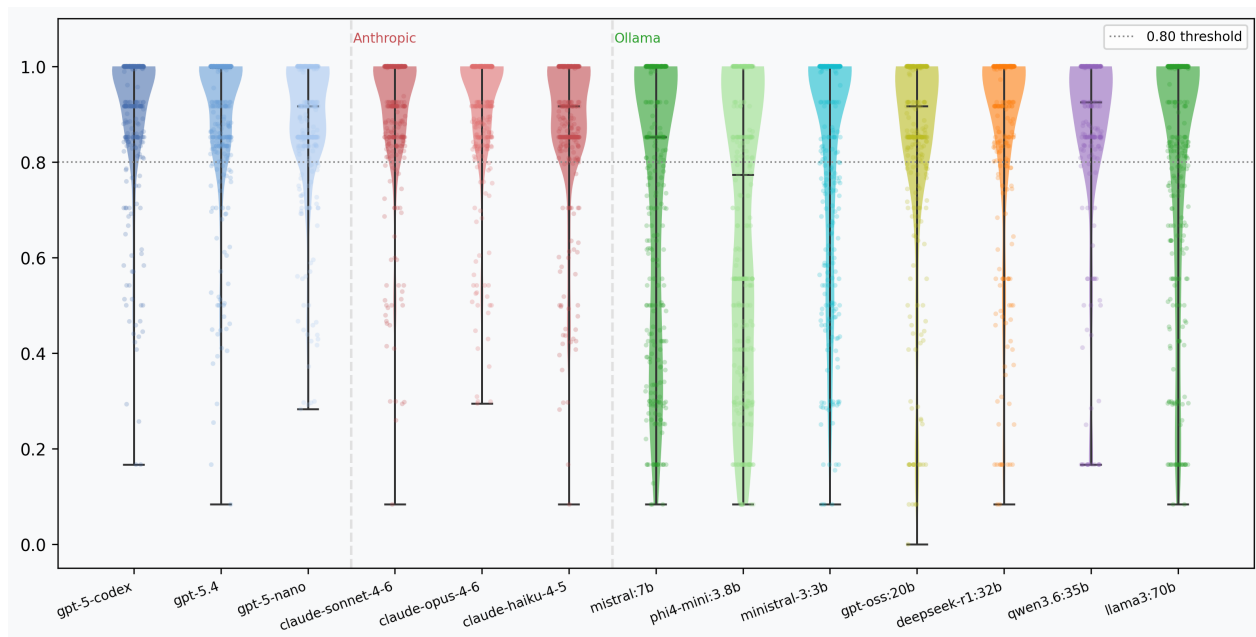


Figure 2: Violin plot depicting individual prompt performance scores

Further, Figure 2 depicts score distribution of all individual task prompt in a *violin plot*. As expected, frontier models from OpenAI and Anthropic show tight, top heavy distributions, most scores cluster near 1.0 with thin tails, indicating consistent high performance. In contrast, open source models show the most diverse patterns: `mistral:7b` and `phi4-mini:3.8b` have very wide violins, depicting scores spread heavily from near 0 to 1.0, which indicates high variance across tasks. `ministral-3:3b` is notably the weakest having fat body centered around 0.9 but a long thin tail reaching near 0. Rest of the models have tighter distributions with most mass above 0.80, comparable to commercial models.

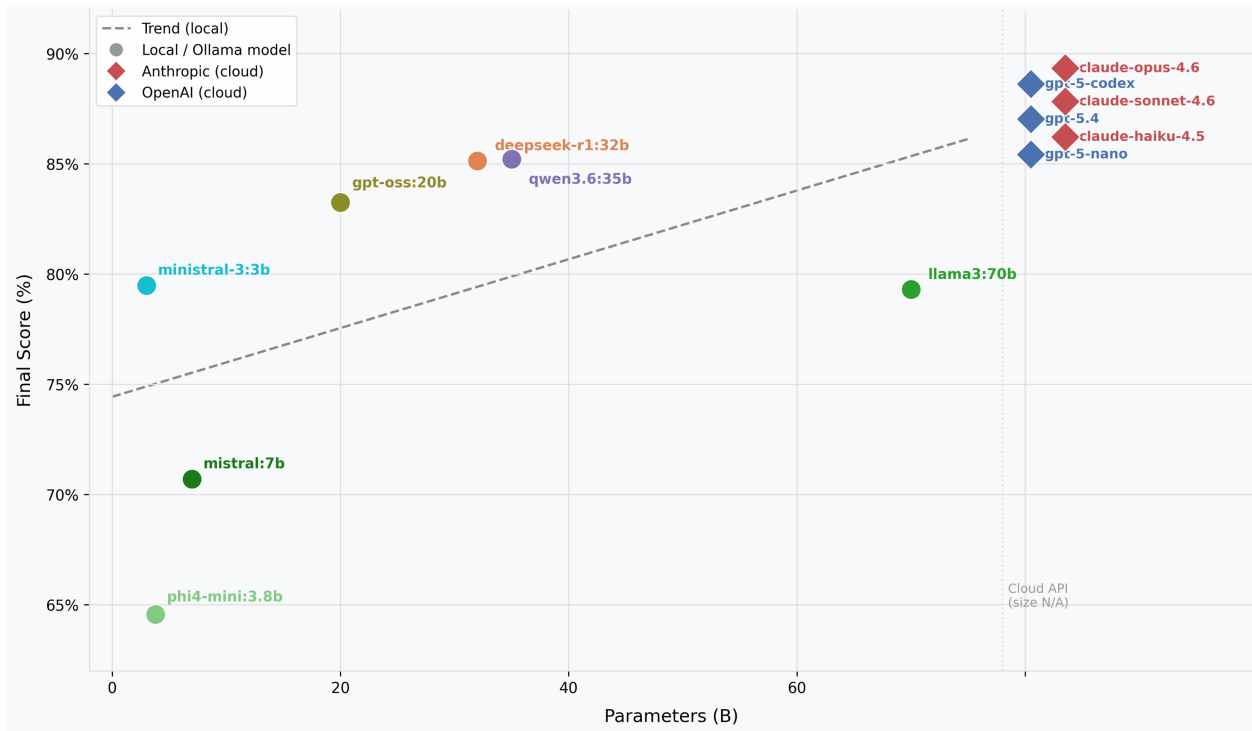


Figure 3: Scatter plot of model performance scores vs. model size (in billions of parameters)

Scatter plot analysis of performance and model size shown in Figure 3 revealed a positive correlation, with larger models generally achieving higher scores. However, some smaller open source models outperformed larger ones. For example, a 3-billion parameter model like `ministral-3`, outperforms a relatively larger but older model like `llama3` with 70 billion parameters, suggesting that parameter count alone is not a sufficient predictor of performance and that architectural innovations and training data quality also play critical roles.

Also, comparing the prompts of best performing open source models to closed source frontier models, we notice that the former often failed to follow instructions, which may contribute to their lower scores on dimensions like code consistency and linting. For example, below code snippet for a code generation task generated by `claude-opus-4.6` on code generation benchmark

```

from typing import List

def filter_by_prefix(strings: List[str], prefix: str) -> List[str]:
    return [s for s in strings if s.startswith(prefix)]

```

In contrast, the same prompt generated by `qwen3.6` produced the following code snippet:

```

return [s for s in strings if s.startswith(prefix)]

```

The task itself is simple and straightforward, but the second snippet fails to follow the instruction to define a function with the specified signature, which likely contributed to its lower score on functional correctness and code consistency dimensions. This pattern of instruction-following failure was observed across multiple tasks and models, suggesting that it may be a common weakness in open source models compared to their closed source counterparts.

5.4 Task Performance Breakdown

As shown in Figure 4, task difficulty varied meaningfully across the six benchmark categories. Models performed most confidently on Code Generation and Translation as they are well defined with clear input output expectations, which likely explains both their high scores and low variance. Code Review similarly exceeded the quality threshold, suggesting LLMs have internalized common code quality heuristics reasonably well. Bug Fixing crossed the threshold on average but showed high disagreement between models, implying that bug complexity significantly influences whether a model succeeds or fails. Refactoring and Test Generation fell below the threshold entirely, these tasks require deeper semantic understanding of intent and behavior, not just syntactic manipulation, which appears to be where current LLMs most consistently fall short. The high variance in Test Generation further suggests that success

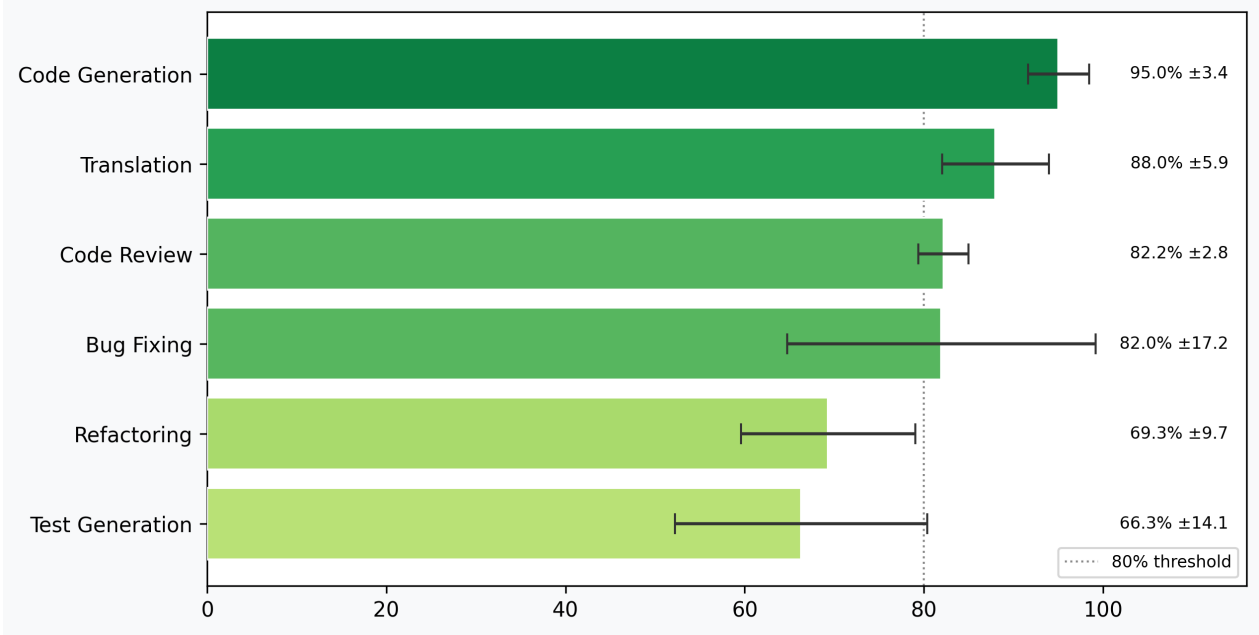


Figure 4: Mean task performance scores across all models

here is inconsistent, likely dependent on how familiar the model is with the specific coding patterns presented.

5.5 Comparison

Due to the lack of a unified evaluation framework and rapidly evolving LLMs, direct comparison of our results to existing benchmarks is challenging. However, we can attempt to contextualize our findings by comparing them to reported scores on LiveCodeBench [18], a code-generation-specific benchmark.

Table 4 presents a side-by-side comparison of FACETS scores with LiveCodeBench results for the closed-source frontier models evaluated. Our framework scores are consistently higher across all tested models, with differences ranging from 2.23% to 25.35%. The Opus model shows the closest alignment between frameworks, while larger disparities are observed for

Table 4: FACETS vs. LiveCodeBench

Model Name	FACETS Score	LiveCodeBench Score	Difference
claude-opus-4.6	89.33%	87.1%	+2.23
claude-sonnet-4.6	88.40%	71.4%	+16.90
claude-haiku-4.6	86.85%	61.5%	+25.35
gpt-5-codex	88.62%	84.0%	+4.62

smaller models like Haiku and Sonnet. This variation suggests that our benchmarking framework may weight different dimensions of code quality differently than LiveCodeBench, or that the curated datasets and evaluation criteria in our framework capture aspects of code generation performance that produce more consistent relative rankings across model tiers.

6 Discussion and Limitations

The results of this evaluation reveal a consistent and interpretable pattern: LLMs perform well when tasks are structurally constrained, and struggle when tasks demand deeper reasoning about code behavior and intent. This distinction is not merely a performance gap, it reflects a fundamental difference in what current language models are capable of. Tasks like code generation and translation can often be solved by pattern recognition and surface-level understanding of syntax, which is where LLMs excel. In contrast, tasks like refactoring and test generation require a more robust internal model of code semantics, the ability to reason about how changes affect behavior, and an understanding of software engineering principles, which are areas where LLMs consistently underperform.

One remarkable observation in open-source models was that the models dominating socreboards like `qwen3.6` and `deepseek-r1` were reasoning models and consistently reasoned before providing an answer, which likely contributed to their higher scores compared to other open

source models. This suggests that architectural innovations that enhance reasoning capabilities may be a key pathway for improving LLM performance on complex software engineering tasks.

Another observable pattern is that worst performance recorded was just shy of 65%, while most of the frontier models clustered above 85%, and the best open source models clustered around 80%. Score clusters are a known limitation of our framework, which can be occurring due to a variety of factors including the datasets not being sufficiently challenging to differentiate between models, or the evaluation dimensions not being sensitive enough to capture nuanced differences in code quality. With rapidly changing landscape of LLMs, it is possible that newer frontier models are now so powerful that they are able to solve most of the benchmark tasks with high consistency, leading to a ceiling effect in the scores.

Furthermore, the dataset samples we use are small chunks of code, which by no means represent the full spectrum of software engineering tasks. Real world codebases are much larger, more complex, and have richer contextual dependencies than the snippets we evaluate. This limits the external validity of our findings and suggests that performance on our benchmarks may not fully predict real world performance. Although, the modular design of our framework allows us to easily implement testing for `javascript` and `cpp` code snippets, we have currently only implemented the evaluation dimensions for `python`. This is a significant acknowledged limitation as well.

7 Conclusion and Future Work

Despite these limitations, FACETS provides a comprehensive and nuanced evaluation of code generation capabilities across multiple dimensions of quality. It reveals meaningful differences between models that are not captured by existing benchmarks, and highlights specific areas where current LLMs excel or struggle. Our findings suggest that while LLMs have made impressive strides in generating syntactically correct code, they still lack a robust understanding of code semantics and intent, which is critical for tasks like refactoring and test generation.

Future work will focus on expanding the benchmark suite to include more complex and realistic coding tasks, such as multi-file projects, integration testing, and codebase navigation. We also plan to refine our existing datasets to better capture the nuances of real world coding scenarios. Additionally, we aim to explore the impact of different prompting strategies and model architectures on performance across these benchmarks, with the goal of identifying pathways for improving LLM capabilities in software engineering applications.

References

- [1] Kreyszig, Erwin. Introductory Functional Analysis with Applications. Wiley Classics Library. 1989.
- [2] Cawood, M.E., V.J. Ervin, W.J. Layton, and J.M. Maubach. Adaptive defect correction methods for convection dominated, convection diffusion problems. *Journal of Computational and Applied Mathematics*. 200. 1-21.

- [3] Sathvik Joel and Jie JW Wu and Fatemeh H. Fard
A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages
arXiv:2410.03981 [cs.SE] 2025. <https://arxiv.org/abs/2410.03981>
- [4] Ruisheng Cao and Mouxiang Chen and Jiawei Chen and Zeyu Cui and Yunlong Feng and
Binyuan Hui and Yuheng Jing and Kaixin Li and Mingze Li and Junyang Lin and Zeyao Ma and
Kashun Shum and Xuwu Wang and Jinxi Wei and Jiayi Yang and Jiajun Zhang and Lei Zhang
and Zongmeng Zhang and Wenting Zhao and Fan Zhou. Qwen3-Coder-Next Technical Report
arXiv:2603.00729 [cs.CL] 2026. <https://arxiv.org/abs/2603.00729>
- [5] GLM-5-Team. GLM-5: from Vibe Coding to Agentic Engineering arXiv:2602.15763 [cs.LG]
2026. <https://arxiv.org/abs/2602.15763>
- [6] Kimi Team. Kimi K2.5: Visual Agentic Intelligence arXiv:2602.02276 [cs.CL] 2026.
<https://arxiv.org/abs/2602.02276>
- [7] OpenAI. GPT-5.3-Codex System Card <https://deploymentsafety.openai.com/gpt-5-3-codex/gpt-5-3-codex.pdf> Accessed: 2026-04-11.
- [8] Anthropic. Claude Opus 4.6 System Card <https://www.anthropic.com/claude-opus-4-6-system-card> 2026. Accessed: 2026-04-11.
- [9] Google DeepMind. Gemini 3.1 Pro Model Card <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Pro-Model-Card.pdf> 2026 Accessed: 2026-04-11.
- [10] Shuo Ren and Daya Guo and Shuai Lu and Long Zhou and Shujie Liu and Duyu Tang and Neel Sundaresan and Ming Zhou and Ambrosio Blanco and Shuai Ma.
CodeBLEU: a Method for Automatic Evaluation of Code Synthesis arXiv:2009.10297 [cs.SE]
2020. <https://arxiv.org/abs/2009.10297>
- [11] Mark Chen and others. Evaluating Large Language Models Trained on Code arXiv:2107.03374
[cs.LG] 2021. <https://arxiv.org/abs/2107.03374>

- [12] Jacob Austin and others. Program Synthesis with Large Language Models arXiv:2108.07732 [cs.PL] 2021. <https://arxiv.org/abs/2108.07732>
- [13] Carlos E. Jimenez and others. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] 2023. <https://arxiv.org/abs/2310.06770>
- [14] Abbas Sabra and Olivier Schmitt and Joseph Tyler. Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis arXiv:2508.14727 [cs.SE] 2025. <https://arxiv.org/abs/2508.14727>
- [15] Jiawei Gu and others. A Survey on LLM-as-a-Judge arXiv:2411.15594 [cs.CL] 2024. <https://arxiv.org/abs/2411.15594>
- [16] Mateusz Pawlik and Nikolaus Augsten. RTED: A Robust Algorithm for the Tree Edit Distance arXiv:1805.06869 [cs.CG] 2018. <https://arxiv.org/abs/1805.06869>
- [17] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: a multi-lingual program repair benchmark set based on the quixey challenge. Association for Computing Machinery, New York, NY, USA, 55–56. <https://doi.org/10.1145/3135932.3135941>
- [18] Naman Jain and others. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models arXiv:2403.07974 [cs.SE] 2024. <https://arxiv.org/abs/2403.07974>

Biographical Sketch

Yash Patel is a senior undergraduate student pursuing a Bachelor's degree in Computer Science at Stetson University in DeLand, Florida. His research interests include artificial intelligence, particularly language models and computer vision. His recent work explores a novel application of the Squirrel Search Algorithm for prompt optimization in language models and was presented at ACM Southeast Conference 2026 in Troy, Alabama.